PACKAGED CLI DIAGNOSTICS TOOL

A Developer Environment Health Checker

PROJECT REPORT

ABSTRACT

The Packaged CLI Diagnostics Tool is a Python-based command-line application designed to inspect a computer's developer environment and generate a structured health report. Developers frequently encounter configuration problems caused by incompatible Python versions, insufficient disk space, missing development tools, incorrect environment variables, and malformed configuration paths. Identifying these issues manually can be time-consuming and difficult, especially for beginners.

The proposed tool automates the process of checking essential developer-environment components. It examines the installed Python version, available disk space, selected environment variables, and the availability of configured developer tools. The application analyzes the collected information and produces a deterministic diagnostic report in both human-readable and JSON formats.

The project uses Python for implementation, a pyproject.toml file for package configuration, and a virtual environment for isolated development and testing. A console entry point allows users to execute the tool directly from the command line after installation. Appropriate exit codes indicate whether the environment passes the configured checks or contains errors.

Unit tests are designed to evaluate normal execution, missing dependencies, and malformed configuration paths. The project aims to simplify environment verification, improve troubleshooting efficiency, and provide a reusable diagnostic solution for software developers.

Keywords: Python, Command-Line Interface, Environment Diagnostics, JSON, Virtual Environment, Software Testing, Package Distribution.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Existing System

6. Proposed System
7. System Requirements
8. System Architecture
9. System Design and Modules
10. Implementation
11. Testing and Validation
12. Results and Discussion
13. Advantages
14. Limitations
15. Future Enhancements
16. Conclusion
17. References
18. Appendices

CHAPTER 1: INTRODUCTION

1.1 Introduction

A developer environment consists of software, tools, libraries, configuration files, environment variables, and system resources required for software development. A properly configured environment is essential for executing programs, installing dependencies, and building software applications.

Developers may experience errors when the installed Python version is incompatible with a project, required tools are missing, disk space is insufficient, or environment variables are incorrectly configured. Detecting these issues manually requires multiple commands and technical knowledge.

The Packaged CLI Diagnostics Tool addresses this problem by providing a single command-line application that checks important components of a developer environment and generates a comprehensive health report.

The tool is implemented in Python and distributed as an installable package. It supports both human-readable output and machine-readable JSON output, making it useful for individual developers, development teams, and automated workflows.

1.2 Problem Statement

Developers often spend unnecessary time identifying problems in their development environments. Manual verification of Python versions, disk availability, environment variables, and required developer tools can result in inconsistent checks and incomplete reports.

There is a need for a lightweight, portable, and reusable command-line tool that automatically inspects these components, identifies configuration problems, and generates a standardized diagnostic report with meaningful exit codes.

1.3 Objectives

The primary objectives of the project are:

1. To develop a command-line diagnostic application using Python.
2. To inspect the installed Python version and compare it against configured requirements.
3. To check available disk space and identify insufficient storage.
4. To inspect selected environment variables without unnecessarily exposing sensitive values.
5. To verify the availability of configured developer tools.
6. To validate configuration files and detect malformed paths.
7. To generate deterministic JSON and human-readable reports.
8. To provide meaningful process exit codes for success and failure conditions.
9. To package the application using pyproject.toml.
10. To create an isolated virtual environment for development and testing.
11. To implement unit tests for successful execution and common error scenarios.
12. To document installation, execution, configuration, and troubleshooting procedures.

1.4 Scope of the Project

The project focuses on local developer-environment diagnostics.

The application can be configured to inspect Python compatibility, disk space, environment-variable presence, and selected development tools such as Git and Python package-management utilities.

The system produces reports that help users understand the current state of their development environment.

The project does not automatically repair system configuration, install missing tools, modify operating-system settings, or guarantee compatibility with every software project. Its primary purpose is inspection and reporting.

CHAPTER 2: EXISTING SYSTEM

2.1 Existing System Description

In the existing approach, developers manually execute commands to inspect Python versions, available storage, environment variables, and installed development tools.

For example, developers may use terminal commands to determine whether Python or Git is installed and inspect system settings to identify configuration issues.

Although this approach can provide the necessary information, it requires users to know which commands to execute and how to interpret their results.

2.2 Limitations of the Existing System

The major limitations are:

1. Manual execution of multiple commands.
2. Lack of a unified diagnostic report.
3. Possibility of overlooking configuration errors.
4. Difficulty comparing results across different machines.
5. Lack of standardized machine-readable output.
6. Inconsistent handling of missing dependencies.
7. Limited integration with automated testing and development workflows.

2.3 Need for the Proposed System

An automated diagnostic application can combine multiple checks into one execution. It can present the results in a consistent format, identify common problems, and return appropriate exit codes.

This approach reduces repetitive manual work and makes environment verification easier for developers.

CHAPTER 3: PROPOSED SYSTEM

3.1 Proposed System Description

The proposed system is a packaged Python command-line tool that performs a series of diagnostic checks and generates a developer-environment health report.

When the user executes the CLI command, the application collects relevant system information, validates the configured requirements, evaluates each diagnostic check, and generates a report.

The report can be displayed in a human-readable format or exported as JSON for further processing.

The application is designed to provide predictable output for equivalent diagnostic inputs, subject to the stability of the underlying machine state and explicitly defined reporting rules.

3.2 Features of the Proposed System

- Python version inspection.
- Disk-space availability checks.
- Environment-variable validation.
- Developer-tool availability checks.
- Configuration-path validation.
- Human-readable diagnostic output.
- Structured JSON output.
- Meaningful exit codes.
- Installable Python package.
- Isolated virtual-environment support.
- Automated unit testing.
- Usage documentation and sample reports.

3.3 System Workflow

The system follows the workflow given below:

1. The user installs the CLI package.
2. The user executes the diagnostic command.
3. The application loads the configuration.
4. The system validates the configuration and its paths.
5. The application inspects the Python version.

6. The application checks available disk space.
7. The application verifies selected environment variables.
8. The application checks the availability of configured developer tools.
9. The diagnostic results are collected and classified.
10. The application generates a human-readable or JSON report.
11. The program returns an exit code based on the overall result.

CHAPTER 4: SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor or equivalent
RAM	4 GB
Storage	Sufficient free space for Python and project files
Display	Standard monitor or laptop display
Input Device	Keyboard
Output Device	Monitor or terminal output

These are proposed development requirements and may be adjusted according to the actual implementation and testing environment.

4.2 Software Requirements

Software	Purpose
Operating System	Windows, Linux, or macOS, subject to compatibility
Python	Application development and execution
pip	Package installation
venv	Isolated Python environment
Git	Version control and optional diagnostic check
Code Editor	Writing and debugging source code

Software	Purpose
pytest	Automated unit testing
pyproject.toml	Package and build configuration
JSON	Structured diagnostic reporting

4.3 Programming Language

Python is selected because it provides built-in modules for inspecting system information, checking executable availability, handling JSON data, and managing file paths.

Its readable syntax and extensive standard library make it suitable for developing a lightweight diagnostic application.

4.4 Development Tools

The project uses a Python virtual environment to isolate its development dependencies from the global Python installation.

The package is configured through pyproject.toml, which defines its metadata, dependencies, build requirements, and command-line entry point.

The pytest framework is used to automate the execution of unit tests.

CHAPTER 5: SYSTEM ARCHITECTURE

5.1 Architecture Overview

The proposed application follows a modular architecture consisting of the following components:

1. Command-Line Interface Module
2. Configuration Management Module
3. Environment Inspection Module
4. Diagnostic Evaluation Module
5. Report Generation Module
6. Exit-Code Management Module
7. Unit Testing Module

5.2 Architecture Description

User → Command-Line Interface → Configuration Loader → Diagnostic Modules → Result Evaluator → Report Generator → Terminal/JSON Output

The command-line interface accepts user options. The configuration loader reads and validates the selected configuration. The diagnostic modules inspect the system and return individual results.

The result evaluator combines the diagnostic results and determines the overall status. The report generator formats the results for terminal display or JSON output. Finally, the application returns an exit code representing the outcome.

5.3 Data Flow

The input consists of command-line arguments, configuration settings, and information collected from the local system.

The processing stage performs system checks and evaluates the results against configured requirements.

The output consists of a diagnostic report containing check names, statuses, messages, and relevant details, along with an appropriate process exit code.

CHAPTER 6: SYSTEM DESIGN AND MODULES

6.1 Command-Line Interface Module

This module provides the main interface between the user and the application.

It accepts command-line arguments such as the output format and configuration-file path.

Possible commands include:

envdoctor

envdoctor --format json

envdoctor --config config.json

envdoctor --help

The exact options depend on the implemented CLI.

6.2 Configuration Management Module

This module loads the user-defined configuration and checks whether it is valid.

It verifies that the configuration file exists, can be read, and contains the expected structure.

The module also validates configured file paths and reports malformed or inaccessible paths appropriately.

Invalid configuration should produce a clear diagnostic message instead of an unhandled exception.

6.3 Python Version Inspection Module

This module identifies the Python interpreter version currently running the application.

The version is compared with the minimum or supported version specified in the configuration.

If the installed version does not satisfy the requirement, the module reports a failure or warning according to the defined rules.

6.4 Disk-Space Inspection Module

This module determines the available storage space on a configured path or relevant filesystem.

The available space is compared against a configured minimum threshold.

If the available space is insufficient, the module records the result and provides a message indicating the detected condition.

6.5 Environment-Variable Inspection Module

This module checks whether required environment variables are available.

It can identify missing or empty variables and report their status.

Sensitive environment-variable values should not be printed in the report unless there is a clear and justified requirement. Reporting variable names and presence status is generally safer.

6.6 Developer-Tool Inspection Module

This module checks whether configured developer tools are available on the system.

Examples include:

- Python
- Git
- pip

Additional tools can be included according to the project configuration.

The module uses executable-discovery mechanisms to determine whether the configured tools can be located.

A missing required tool is reported as a diagnostic failure.

6.7 Report Generation Module

The report generation module converts diagnostic results into two output formats.

Human-readable report: Displays check names, statuses, and messages in a format that users can understand easily.

JSON report: Provides structured output suitable for scripts, automated workflows, and further analysis.

The JSON format should use a consistent schema so that downstream applications can interpret the results reliably.

6.8 Exit-Code Management Module

The exit-code module communicates the overall diagnostic outcome to the operating system or calling process.

A proposed exit-code scheme is:

Exit Code	Meaning
0	All required checks passed
1	One or more required diagnostic checks failed
2	Invalid command-line arguments or malformed configuration
3	Unexpected internal error

These values are a proposed convention. The final implementation must use the same mapping consistently throughout the application and documentation.

6.9 Unit Testing Module

This module verifies that the application's components work as expected.

Tests are written for normal execution, missing dependencies, invalid configuration, and malformed configuration paths.

Tests should isolate external system conditions through mocking wherever practical, allowing reliable and repeatable test execution.

CHAPTER 7: IMPLEMENTATION

7.1 Project Directory Structure

The project repository is organized as follows:

```
EnvDoctor/
│
├── pyproject.toml
├── README.md
├── .gitignore
├── config.example.json
│
├── src/
│   └── envdoctor/
│       ├── __init__.py
│       ├── cli.py
│       ├── config.py
│       ├── diagnostics.py
│       └── reports.py
│
├── tests/
│   ├── test_cli.py
│   ├── test_diagnostics.py
│   └── test_config.py
│
├── examples/
│   ├── sample-report.txt
│   └── sample-report.json
│
└── docs/
    └── usage.md
```

The structure separates application source code, automated tests, sample reports, configuration examples, and documentation.

The exact directory structure may be modified to match the final implementation.

7.2 Virtual Environment Setup

A virtual environment is created to isolate project dependencies.

The following command creates the environment:

python -m venv .venv

On Windows PowerShell, the environment can be activated using:

.\.venv\Scripts\Activate.ps1

On Windows Command Prompt, use:

.venv\Scripts\activate.bat

On Linux or macOS, use:

source .venv/bin/activate

If PowerShell blocks the activation script because of its execution policy, the project can alternatively be operated using the virtual environment's Python executable directly. Activation is convenient but not strictly necessary.

7.3 Package Configuration

The pyproject.toml file contains the package metadata and installation configuration.

It specifies the package name, version, Python requirements, build system, and console entry point.

A console entry point connects the installed command to the application's main function. After installation, the user can execute the tool using its configured command name.

The package configuration should also include the required build-system settings and dependencies for reproducible installation.

7.4 Environment Inspection

The inspection module collects the required system information.

It determines the Python version, examines available disk space, checks required environment variables, and locates configured developer tools.

Each inspection function returns a structured result that can be passed to the evaluation and reporting modules.

Separating individual checks improves maintainability and makes unit testing easier.

7.5 Report Generation

The reporting module receives the collected diagnostic results and produces the requested output format.

A typical report contains:

- Report version.
- Overall status.
- Python version information.
- Disk-space status.
- Environment-variable check results.
- Developer-tool availability.
- Configuration validation results.
- Diagnostic messages.

For deterministic reporting, the application should use a stable ordering for checks and JSON keys, consistent status values, and predictable formatting. Volatile values such as timestamps should be excluded from deterministic comparisons or handled explicitly.

7.6 Error Handling

The application handles expected errors such as missing configuration files, invalid JSON, unavailable tools, and inaccessible paths.

Errors should be converted into understandable diagnostic messages.

The application should avoid exposing secret environment-variable values or unnecessary sensitive system information.

Unexpected internal errors should be handled separately from ordinary diagnostic failures.

7.7 Package Installation and Execution

The package is installed from the project directory using:

python -m pip install -e .

The editable installation is useful during development because source-code changes can be tested without reinstalling the package each time.

After installation, the application can be executed using:

envdoctor

JSON output can be requested using:

envdoctor --format json

A configuration file can be supplied using:

envdoctor --config config.example.json

The actual command-line options must match the implemented application.

7.8 Creating a Distributable Package

After verifying the tests and package configuration, the project can be built into distributable Python package files.

A build frontend such as the build package can be used to generate a wheel and source distribution.

The resulting artifacts should be checked for successful installation in a clean virtual environment before being considered ready for distribution.

CHAPTER 8: TESTING AND VALIDATION

8.1 Introduction to Testing

Testing verifies that the diagnostic application produces correct results under normal conditions and handles common errors appropriately.

Unit testing is used to evaluate individual modules, while integration testing checks whether the CLI, configuration loader, diagnostic modules, and report generator work together.

8.2 Testing Objectives

The objectives of testing are:

1. To verify successful application execution.
2. To confirm the correct identification of missing developer tools.
3. To validate malformed configuration handling.
4. To verify the correctness of generated reports.
5. To check the consistency of exit codes.
6. To ensure that errors are handled without unexpected crashes.
7. To verify that the package can be installed and executed.

8.3 Test Cases

Test Case ID	Test Scenario	Input or Condition	Expected Result
TC01	Successful execution	Valid configuration and all required tools available	Successful report and exit code 0
TC02	Missing dependency	A required developer tool is unavailable	Missing-tool diagnostic and nonzero exit code
TC03	Malformed configuration	Configuration contains invalid JSON	Clear configuration error and appropriate exit code
TC04	Missing configuration file	Supplied configuration path does not exist	Configuration-path error
TC05	Python version check	Python version satisfies the configured requirement	Python check passes
TC06	Python version failure	Python version does not satisfy the configured requirement	Version failure is reported
TC07	Disk-space check	Available disk space meets the configured threshold	Disk-space check passes
TC08	Insufficient disk space	Available disk space is below the configured threshold	Disk-space warning or failure
TC09	Missing environment variable	A required environment variable is unavailable	Missing-variable diagnostic
TC10	JSON output	User requests JSON format	Valid structured JSON is generated
TC11	Human-readable output	User requests standard output	Readable diagnostic report is generated
TC12	Malformed path	Configured path is invalid or unusable	Path validation error is reported

The expected outcomes must be verified against the actual implementation and the project's chosen error-handling rules.

8.4 Unit Testing

Unit tests are implemented using pytest.

Individual diagnostic functions are tested with controlled inputs. Mocking can be used to simulate unavailable executables, different Python versions, disk-space conditions, and filesystem errors.

This approach reduces dependence on the machine used to execute the tests.

8.5 Integration Testing

Integration tests verify that the complete application behaves correctly when executed through its command-line entry point.

These tests confirm that configuration loading, diagnostic execution, report generation, and exit-code handling operate together correctly.

8.6 Validation of JSON Output

The JSON output is validated by parsing the generated content using a JSON parser.

The test verifies that the report contains the required fields and that each diagnostic result uses the expected data structure.

8.7 Test Results

The following table should be completed after executing the actual test suite.

Test Case ID	Actual Result	Status
TC01	[Enter observed result]	[Pass/Fail]
TC02	[Enter observed result]	[Pass/Fail]
TC03	[Enter observed result]	[Pass/Fail]
TC04	[Enter observed result]	[Pass/Fail]
TC05	[Enter observed result]	[Pass/Fail]
TC06	[Enter observed result]	[Pass/Fail]
TC07	[Enter observed result]	[Pass/Fail]
TC08	[Enter observed result]	[Pass/Fail]
TC09	[Enter observed result]	[Pass/Fail]
TC10	[Enter observed result]	[Pass/Fail]

Test Case ID	Actual Result	Status
TC11	[Enter observed result]	[Pass/Fail]
TC12	[Enter observed result]	[Pass/Fail]

Note: Do not mark test cases as passed until they have actually been executed and verified.

CHAPTER 9: RESULTS AND DISCUSSION

9.1 Results

The Packaged CLI Diagnostics Tool is designed to inspect important components of a developer environment and present the results through a standardized report.

The expected output includes the Python version, available disk space, environment-variable status, developer-tool availability, and configuration-validation results.

The application supports both terminal output and structured JSON output.

After implementation and testing, this section should document the actual results obtained from the system.

9.2 Sample Human-Readable Report

The following is an illustrative report. Replace it with output generated by the actual application.

```
========================================
    ENVDOCTOR HEALTH REPORT
========================================


Overall Status : PASS


Python Version : PASS
Disk Space     : PASS
Environment    : PASS
Developer Tools: PASS
Configuration  : PASS
```

Summary:

All required diagnostic checks passed.

Exit Code: 0

==========

9.3 Sample JSON Report

The following is an illustrative example of the intended report structure.


```
{
  "report_version": "1.0",
  "overall_status": "PASS",
  "checks": [
    {
      "name": "python_version",
      "status": "PASS",
      "message": "Python version satisfies the requirement."
    },
    {
      "name": "disk_space",
      "status": "PASS",
      "message": "Available disk space meets the configured threshold."
    },
    {
      "name": "environment_variables",
      "status": "PASS",
      "message": "Required environment variables are available."
    },
    {
```

```
    "name": "developer_tools",
    "status": "PASS",
    "message": "All required developer tools are available."
   },
   {
    "name": "configuration",
    "status": "PASS",
    "message": "Configuration is valid."
   }
 ],
 "exit_code": 0
}
```


The report structure may be adapted to the final implementation. The example does not represent actual test results.

9.4 Discussion

The proposed tool consolidates several environment checks into a single application. It reduces the need to execute separate diagnostic commands and provides consistent output for users and automated scripts.

JSON reporting enables the results to be processed by other applications, while human-readable reporting helps developers understand configuration problems quickly.

The usefulness and reliability of the tool depend on the correctness of its diagnostic functions, the quality of its configuration validation, and the consistency of its error-handling logic.

9.5 Screenshots

Insert actual screenshots of the following outputs in the final report:

Figure 9.1: Project directory structure.

[Insert screenshot here]

Figure 9.2: Virtual environment creation and package installation.

[Insert screenshot here]

Figure 9.3: Successful CLI execution.

[Insert screenshot here]

Figure 9.4: JSON diagnostic report.

[Insert screenshot here]

Figure 9.5: Missing dependency error.

[Insert screenshot here]

Figure 9.6: Malformed configuration error.

[Insert screenshot here]

Figure 9.7: Automated unit-test execution.

[Insert screenshot here]

CHAPTER 10: ADVANTAGES AND LIMITATIONS

10.1 Advantages

1. Automates common developer-environment checks.
2. Reduces repetitive manual verification.
3. Provides consistent diagnostic output.
4. Supports human-readable and machine-readable reports.
5. Uses meaningful exit codes for automation.
6. Supports isolated development and testing.
7. Provides a modular structure for future extensions.
8. Uses unit tests to verify important error conditions.
9. Can be distributed as an installable Python package.
10. Helps developers identify configuration issues earlier.

10.2 Limitations

1. The quality of the report depends on the checks implemented.
2. Some diagnostic checks may behave differently across operating systems.
3. The tool identifies problems but does not automatically repair them.
4. Incorrect configuration requirements may produce misleading failures.
5. Access permissions can restrict the information available to the application.

6. A successful report does not guarantee that every development project will run correctly.
7. External tools may be installed but still be unusable because of incorrect configuration.

CHAPTER 11: FUTURE ENHANCEMENTS

The project can be extended with the following features:

1. **Automatic Repair Suggestions:** Provide safe instructions for resolving detected configuration issues.
2. **Additional Developer Tools:** Support checks for Docker, Node.js, Java, compilers, and other tools.
3. **Custom Configuration Profiles:** Allow different projects to define their own environment requirements.
4. **Continuous Integration Support:** Integrate the diagnostic command into automated build and testing pipelines.
5. **HTML Reports:** Generate graphical reports with status indicators and diagnostic summaries.
6. **Historical Reports:** Compare diagnostic results across multiple runs to identify changes.
7. **Cross-Platform Improvements:** Expand and validate support for Windows, Linux, and macOS.
8. **Dependency Compatibility Checks:** Identify incompatible or unsupported dependency versions.
9. **Improved Security:** Redact sensitive environment information and provide configurable privacy controls.
10. **Automated Test Expansion:** Increase test coverage for unusual filesystem conditions and configuration errors.

These enhancements could make the application more useful for individual developers and larger software teams.

CHAPTER 12: CONCLUSION

The Packaged CLI Diagnostics Tool demonstrates how Python can be used to automate the inspection of a developer environment through a command-line interface.

The application is designed to examine the Python version, disk-space availability, environment variables, configured developer tools, and configuration paths. It generates human-readable and JSON reports, enabling users to understand diagnostic results and integrate them into automated workflows.

The use of pyproject.toml, virtual environments, a console entry point, and automated unit tests provides a structured approach to developing and distributing a reusable Python application.

The project also emphasizes modular design, predictable reporting, input validation, error handling, and meaningful exit codes. These characteristics make the tool a useful foundation for improving developer-environment verification.

The completed project can serve as a practical example of Python packaging, command-line application development, system inspection, structured reporting, and software testing.

The project objectives should be considered achieved only after the application has been implemented, its tests have been executed successfully, and the generated reports have been validated.

REFERENCES

1. Python Software Foundation, *Python Documentation*. Available at: https://docs.python.org/3/
2. Python Packaging Authority, *Python Packaging User Guide*. Available at: https://packaging.python.org/
3. Python Software Foundation, *venv — Creation of Virtual Environments*. Available at: https://docs.python.org/3/library/venv.html
4. Python Software Foundation, *argparse — Parser for Command-Line Options*. Available at: https://docs.python.org/3/library/argparse.html
5. Python Software Foundation, *json — JSON Encoder and Decoder*. Available at: https://docs.python.org/3/library/json.html
6. pytest Development Team, *pytest Documentation*. Available at: https://docs.pytest.org/
7. Python Software Foundation, *pathlib — Object-Oriented Filesystem Paths*. Available at: https://docs.python.org/3/library/pathlib.html

8. Python Software Foundation, *shutil — High-Level File Operations*. Available at: https://docs.python.org/3/library/shutil.html

---

APPENDIX A: SAMPLE COMMANDS

The following commands illustrate the intended usage of the application.

1. Create a virtual environment

python -m venv .venv

2. Install the package in editable mode

python -m pip install -e .

3. Run the diagnostic tool

envdoctor

4. Generate JSON output

envdoctor --format json

5. Run with a configuration file

envdoctor --config config.example.json

6. Display help

envdoctor --help

7. Run unit tests

python -m pytest -v

8. Build distributable package files

python -m pip install build

python -m build

These commands should be adjusted if the implemented CLI uses different options or command names.

APPENDIX B: SAMPLE CONFIGURATION

The following is an illustrative JSON configuration. The application must implement the corresponding configuration schema before this example can be used.

{

```
 "python": {
  "minimum_version": "3.10"
 },
 "disk": {
  "path": ".",
  "minimum_free_mb": 500
 },
 "environment_variables": [
  "PATH"
 ],
 "developer_tools": [
  "python",
  "git"
 ]
}
```

The configuration defines a minimum Python version, a disk-space threshold, a list of environment variables to check, and a list of developer tools to locate.

The required values should be adjusted to suit the intended development environment.

APPENDIX C: PROJECT COMPLETION CHECKLIST

Before submitting the project, verify the following items:

- Project source code is complete.
- pyproject.toml is configured correctly.
- Virtual environment setup is documented.
- Console entry point works after installation.
- Python version check is implemented.
- Disk-space check is implemented.
- Environment-variable check is implemented.

- Developer-tool checks are implemented.
- Configuration validation is implemented.
- JSON reporting works correctly.
- Human-readable reporting works correctly.
- Exit codes are documented and consistent.
- Tests cover successful execution.
- Tests cover missing dependencies.
- Tests cover malformed configuration paths.
- All unit tests have been executed.
- Sample reports are included.
- README.md contains installation and usage instructions.
- Screenshots of actual execution are included.
- The package has been built and installation verified.
- Final report has been reviewed and formatted.
- College certificate and declaration are completed.

END OF PROJECT REPORT